

CM3015 Machine Learning and Neural Networks

Mid-term Coursework Report

Network intrusion detection using classical machine learning

Student Name: Vishal Theerthagiri
Student ID: 240285463
Date: July 11, 2026

This report presents a mini-project on binary network intrusion detection using a balanced CICIDS2017-based dataset. The aim is to compare classical machine learning algorithms from the first half of the module, with particular emphasis on understanding the algorithms, implementing one model from scratch, and evaluating the results critically.

1. Abstract

This project compares classical machine learning algorithms for binary network intrusion detection using a balanced CICIDS2017-based dataset. The task is to classify each network flow as benign or malicious. A naive majority-class baseline, a k-nearest neighbours classifier implemented from scratch, decision tree classifiers with different maximum depths, and PCA-assisted model variants were tested. The baseline achieved 50% accuracy and an f1-score of 0 for the attack class, confirming that a non-learning approach is inadequate. The best scratch KNN model achieved an f1-score of approximately 0.977, while the decision tree with maximum depth 10 achieved an f1-score of approximately 0.994. An unconstrained decision tree achieved the highest f1-score of approximately 0.997, but the depth 10 tree was selected for detailed analysis because it offers strong performance with better control over model complexity. PCA reduced the feature space from 52 features to 18 components while retaining about 95.18% of variance. Additional error analysis, sample-size sensitivity testing, cross-validation, ROC/precision-recall analysis, and bootstrap confidence intervals support the reliability of the selected depth 10 decision tree.

2. Introduction

Network intrusion detection is the task of identifying malicious activity within network traffic. Rule-based systems can detect known attacks, but they may struggle when traffic patterns change or when attacks do not exactly match existing signatures. Machine learning offers an alternative approach by learning patterns directly from historical network-flow data.

This project focuses on binary intrusion detection, where each network flow is classified as either benign or malicious. This is a relevant machine learning problem because network-flow datasets usually contain numerical features such as packet lengths, flow duration, inter-arrival times and header information. These features can be used by classification algorithms to learn differences between normal and attack traffic.

The project is grounded in the wider network intrusion detection literature. The dataset used here is derived from CICIDS2017, introduced by Sharafaldin, Habibi Lashkari and Ghorbani (2018), who generated traffic captures and a flow-based feature set for intrusion detection research. Prior benchmarking work on CICIDS2017, such as Maseer et al. (2021), reports that classical tree-based and distance-based models can achieve high performance, but that results vary considerably with preprocessing choices, feature selection and class balancing. This project therefore compares algorithms in a controlled workflow and discusses limitations of using a balanced preprocessed dataset.

3. Background

3.1 k-nearest neighbours

K-nearest neighbours (KNN) is a supervised classification algorithm based on distance. During training, KNN stores the training examples rather than learning explicit model parameters. During prediction, it calculates the distance between a new example and the stored training examples, selects the k closest neighbours and predicts the most

common class among them.

For a test point x and training point x_i with p features, this project uses Euclidean distance: $d(x, x_i) = \sqrt{\sum_j (x_j - x_{ij})^2}$. Because Euclidean distance is affected by feature scale, standardisation is applied before KNN. A limitation of KNN is prediction-time cost: every test example must be compared with stored training examples.

3.2 decision tree classifier

A decision tree classifies examples by repeatedly splitting the data using threshold-based rules. At each split, the algorithm chooses a feature and threshold that reduces class impurity in the resulting child nodes. Scikit-learn decision trees use Gini impurity by default, where $G(t) = 1 - \sum_c p(c | t)^2$.

Decision trees are useful for tabular data because they are fast and interpretable. However, very deep trees may overfit by learning overly specific patterns from the training data. For this reason, this project compares maximum depths of 3, 5, 10 and no depth limit.

3.3 principal component analysis

Principal component analysis (PCA) is an unsupervised dimensionality reduction technique. It transforms the original features into principal components that preserve as much variance as possible. PCA can reduce runtime and remove redundancy between correlated features. However, PCA does not use class labels, so preserving variance does not always preserve the most useful information for classification.

3.4 evaluation metrics

Accuracy measures the proportion of correct predictions. Precision measures how many predicted attacks were truly attacks. Recall measures how many actual attacks were detected. The f1-score combines precision and recall. In this project, the positive class is attack traffic, so false negatives are particularly important because they represent attacks incorrectly classified as benign.

4. Methodology

The methodology follows a supervised classification workflow. The dataset is loaded and inspected, the target variable is separated from the input features, and the dataset is checked for missing values, infinite values, duplicates and class balance. A balanced 60,000-row sample is used to keep runtime manageable, especially for the scratch KNN implementation. The data is split into training and test sets using stratified sampling so that both sets preserve the 50:50 class balance. Feature scaling is applied before KNN and PCA because both methods are sensitive to feature magnitude.

experiment	purpose	model / technique
baseline	minimum benchmark	majority-class prediction
scratch KNN	from-scratch implementation	$k = 3, 5, 7$
decision tree depth sweep	test complexity and overfitting	$\text{max_depth} = 3, 5, 10, \text{None}$
cross-validation	check stability	depth 10 decision tree, 5 folds
PCA experiment	test dimensionality reduction	PCA retaining 95% variance
threshold analysis	inspect operating thresholds	ROC and precision-recall curves
error analysis	inspect misclassified flows	false positives and false negatives

4.1 dataset preparation

item	value
rows	851388
columns	53
feature columns	52
target column	Attack_Binary
missing values	0
duplicate rows	8

item	value
original class 0	425694
original class 1	425694
sample size	60000
train size	42000
test size	18000

The original dataset is balanced with 425,694 benign and 425,694 attack examples. The experimental sample contains 30,000 examples from each class. The final train/test split is 42,000 training rows and 18,000 test rows.

4.2 scratch KNN implementation

The KNN classifier below was implemented from scratch using standard Python and NumPy rather than a machine learning library. A smaller subset of 5,000 training examples and 2,000 test examples is used for scratch KNN because each test example is compared against all stored training examples.

```
class ScratchKNN:
    def __init__(self, k=5):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.asarray(X)
        self.y_train = np.asarray(y).astype(int)

    def predict_one(self, x):
        distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
        nearest_indices = np.argsort(distances)[:self.k]
        nearest_labels = self.y_train[nearest_indices]
        class_counts = np.bincount(nearest_labels)
        return np.argmax(class_counts)

    def predict(self, X):
        X = np.asarray(X)
        predictions = [self.predict_one(row) for row in X]
        return np.array(predictions)
```

4.3 decision tree, PCA and robustness checks

Decision trees were trained with maximum depths of 3, 5, 10 and no limit. A depth 10 model was selected for detailed analysis because it achieved very high performance while remaining more controlled than the unconstrained tree. Five-fold cross-validation was used to check whether the depth 10 result was stable across different splits. PCA was then used to reduce the standardised feature space while retaining 95% of variance. Finally, sample-size sensitivity, error analysis, threshold analysis and a bootstrap confidence interval were used as robustness checks.

The depth 10 decision tree achieved mean cross-validation $f1 = 0.9935$ with standard deviation 0.0008. PCA reduced the feature space from 52 features to 18 components while retaining 0.9518 of variance.

5. Results

5.1 overall model comparison

model	accuracy	precision	recall	f1_score	runtime_seconds
decision tree depth=None	0.9971	0.9974	0.9968	0.9971	1.0602
decision tree depth=10	0.9936	0.9975	0.9897	0.9936	0.8843
decision tree depth=5	0.9839	0.9868	0.9809	0.9838	0.5703
pca + decision tree depth=10	0.9812	0.9743	0.9886	0.9814	1.1629
scratch knn k=3	0.9780	0.9735	0.9815	0.9775	3.7861
scratch knn k=5	0.9770	0.9696	0.9835	0.9765	3.7707
pca + scratch knn k=5	0.9745	0.9666	0.9815	0.9740	0.9854
scratch knn k=7	0.9740	0.9637	0.9835	0.9735	3.6006
decision tree depth=3	0.9468	0.9467	0.9470	0.9468	0.3968
naive majority baseline	0.5000	0.0000	0.0000	0.0000	0.0000

The naive baseline achieved 50% accuracy because the dataset is balanced and the model predicts only one class. Its f1-score for the attack class is 0, showing that accuracy alone is not sufficient for intrusion detection. The scratch KNN models performed strongly, with k=3 achieving the best KNN f1-score of approximately 0.977. The decision tree models achieved the strongest overall performance.

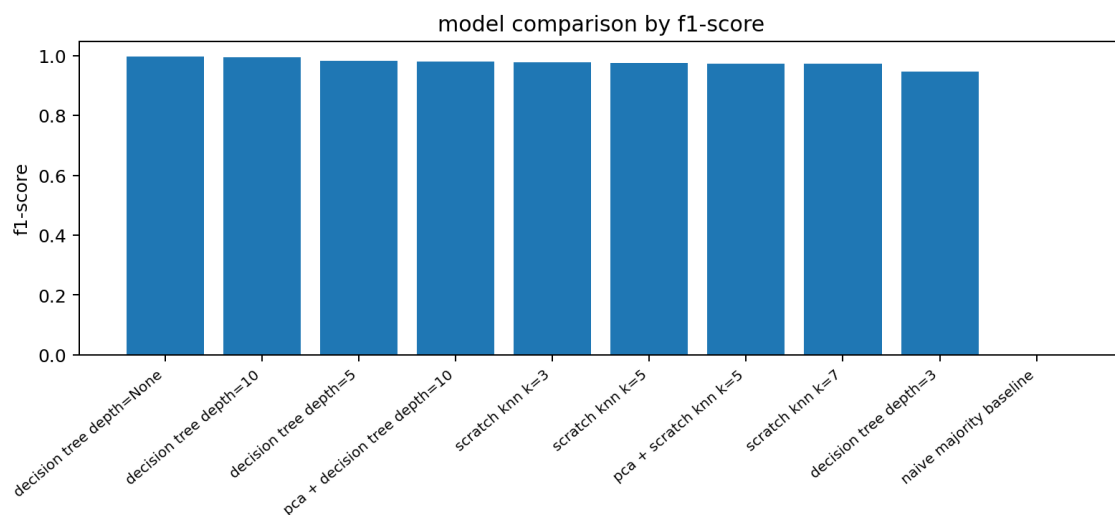


Figure 1. Model comparison by f1-score.

5.2 decision tree analysis

model	accuracy	precision	recall	f1_score	runtime_seconds
decision tree depth=3	0.9468	0.9467	0.9470	0.9468	0.3968
decision tree depth=5	0.9839	0.9868	0.9809	0.9838	0.5703
decision tree depth=10	0.9936	0.9975	0.9897	0.9936	0.8843
decision tree depth=None	0.9971	0.9974	0.9968	0.9971	1.0602

The confusion matrix for the selected depth 10 tree shows 93 false negatives and 22 false positives out of 18,000 test examples. False negatives are the more serious error type in intrusion detection because they represent attack flows classified as benign.

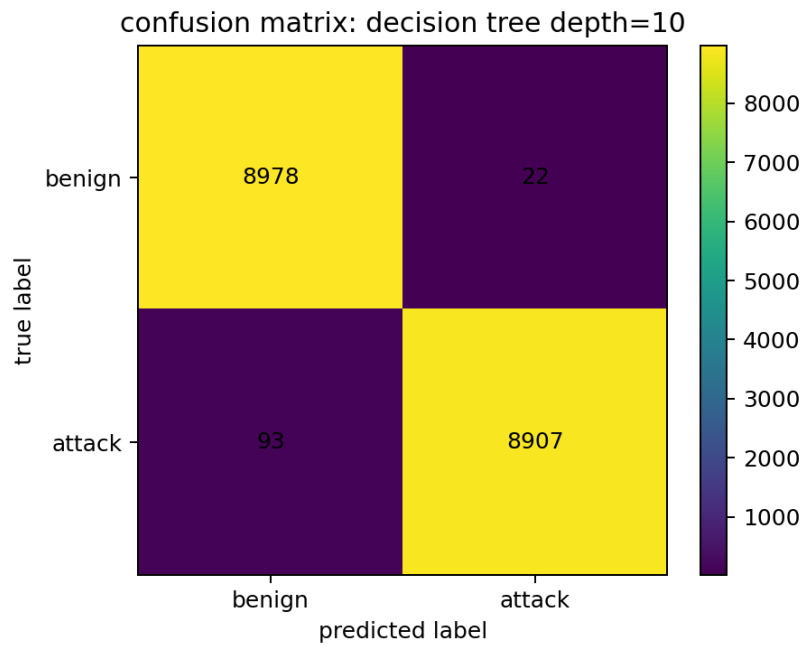


Figure 2. Confusion matrix for the selected depth 10 decision tree.

5.3 robustness checks

The sample-size sensitivity test shows that performance improves slightly as the balanced sample size increases and stabilises at a high level. This supports the use of the 60,000-row sample for the main comparison.

sample_size	accuracy	precision	recall	f1_score	runtime_seconds
5000	0.9900	0.9920	0.9880	0.9900	0.0651
10000	0.9910	0.9966	0.9853	0.9909	0.1402
30000	0.9923	0.9940	0.9907	0.9923	0.4207
60000	0.9936	0.9975	0.9897	0.9936	0.9582

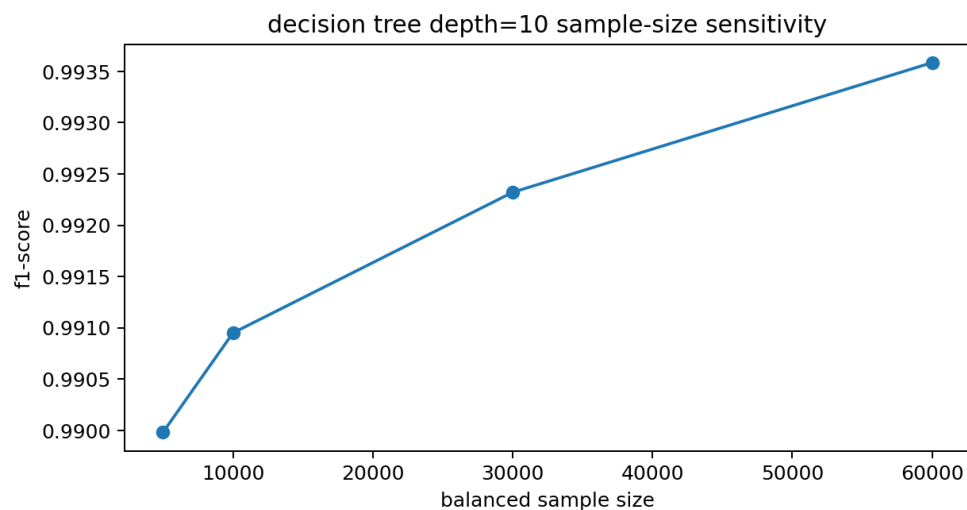


Figure 3. Sample-size sensitivity for the depth 10 decision tree.

The ROC-AUC for the selected decision tree is 0.9984. The bootstrap confidence interval for f1-score is 0.9922 to 0.9947, which indicates that the selected model is stable across resampled versions of the test set.

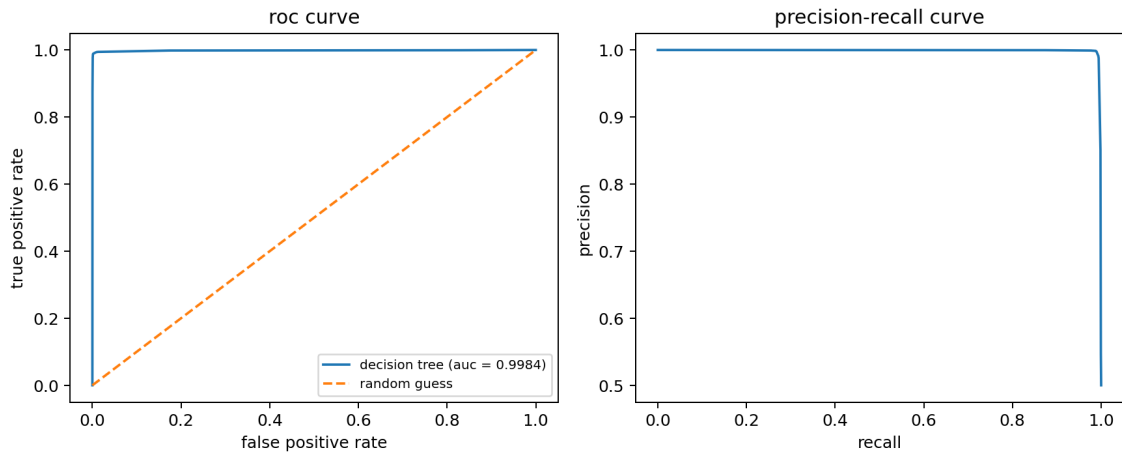


Figure 4. ROC and precision-recall curves for the depth 10 decision tree.

5.4 feature importance and error analysis

feature	importance
Bwd Packet Length Std	0.405485
Average Packet Size	0.290262
Bwd Header Length	0.101156
Destination Port	0.067501
Max Packet Length	0.044958
Init_Win_bytes_forward	0.027865
min_seg_size_forward	0.009691
Bwd Packet Length Max	0.006527
Bwd IAT Std	0.006320
Fwd IAT Mean	0.005350

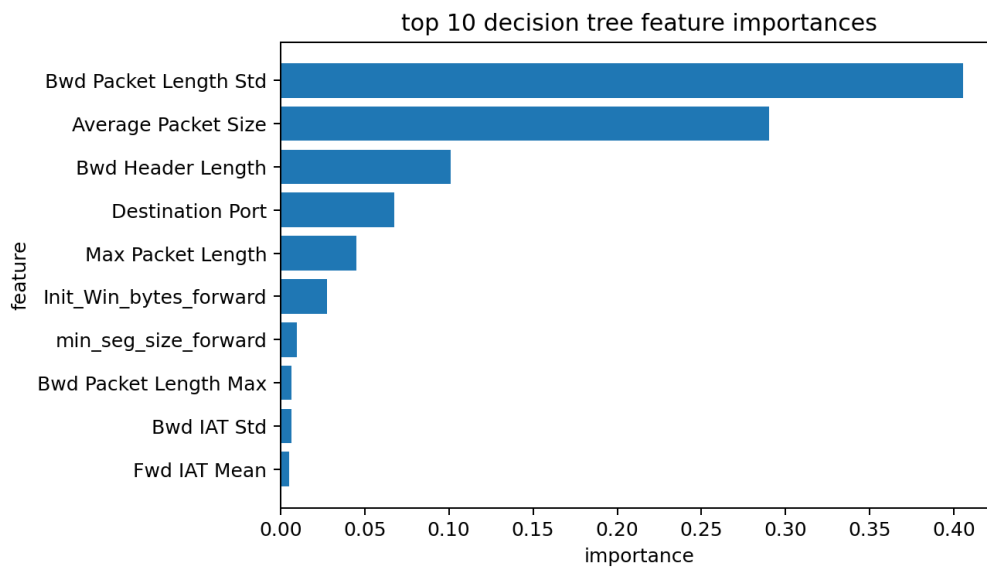


Figure 5. Top 10 decision tree feature importances.

prediction_type	count
true_negative	8978
true_positive	8907
false_negative	93
false_positive	22

The error analysis compares false positives and false negatives using the most important decision tree features. The small number of false negatives shows that the model rarely misses attacks, but those missed attacks remain the most important errors from a security perspective.

prediction_type	Bwd Packet Length Std	Average Packet Size	Bwd Header Length	Destination Port	Max Packet Length
false_negative	190.920	178.018	262.065	7567.903	680.376
false_positive	0.000	6.432	8.727	5575.227	11.545
true_negative	130.228	129.795	164.393	9554.515	519.812
true_positive	1599.649	620.675	119.567	1754.274	3737.072

6. Evaluation

The results show that classical machine learning models can perform strongly on this balanced intrusion detection task. The naive baseline achieved only 50% accuracy and an f1-score of 0 for the attack class, confirming that a simple majority-class strategy is not useful.

The scratch KNN classifier performed well, with the best version reaching an f1-score of approximately 0.977. This is a strong result, especially because the implementation was written from scratch using NumPy rather than a machine learning library. However, KNN was slower than the decision tree because prediction requires distance calculations against stored training examples. This makes KNN less suitable for large-scale real-time intrusion detection unless approximate search or further optimisation is used.

The decision tree was the strongest model family in this project. The unconstrained tree achieved the highest f1-score, but it may be more likely to overfit. The depth 10 decision tree achieved an f1-score of approximately 0.994, which is only slightly lower, while remaining more controlled. The confusion matrix for the depth 10 tree showed 93 false negatives and 22 false positives out of 18,000 test examples. In an intrusion detection context, the false negatives are especially important because they represent attacks incorrectly classified as benign.

PCA reduced the number of features from 52 to 18 while retaining about 95.18% of the variance. This improved the runtime of the KNN experiment, but the f1-score decreased slightly. The PCA decision tree also performed worse than the original depth 10 decision tree. This shows that preserving variance does not always preserve the most useful class-separating information.

The results are broadly consistent with prior classical machine learning literature on CICIDS2017. Maseer et al. (2021) benchmarked 31 anomaly-based intrusion detection models and reported f1-scores predominantly in the 0.90-1.00 range for tree-based classifiers. This suggests that the strong performance obtained here is not simply an artefact of one favourable split, especially since cross-validation and bootstrap testing also support result stability.

There are still several limitations. First, the project used a 60,000-row sample rather than the full dataset to keep runtime manageable. Second, the dataset is already balanced and preprocessed, while real network traffic is usually imbalanced. Third, only classical machine learning algorithms were compared, so the project does not test whether more advanced models such as random forests, gradient boosting or neural networks would improve performance. Finally, because this is a static dataset, it does not fully represent changing attack patterns over time.

Overall, the additional robustness checks strengthen the evaluation. The sample-size sensitivity analysis tested whether the selected sample size was sufficient, while the bootstrap confidence interval gave an estimate of uncertainty around the selected decision tree f1-score. The error analysis separated false positives from false negatives, which is important because these errors have different practical consequences in intrusion detection.

7. Conclusion

This project set out to compare classical machine learning algorithms for binary network intrusion detection. The selected algorithms performed far better than the naive baseline. The scratch KNN classifier demonstrated that distance-based classification can work well, but it is limited by prediction-time cost. The decision tree classifier produced the strongest results, with the depth 10 model achieving an f1-score of approximately 0.994 and strong confusion matrix performance.

PCA reduced the feature space substantially, from 52 features to 18 components, but slightly reduced predictive performance. This suggests that dimensionality reduction can improve efficiency, but it should be evaluated carefully because some discarded or transformed information may still be useful for classification. Overall, the depth 10 decision tree was the best practical model in this project because it combined high accuracy, high f1-score, fast runtime and interpretability.

8. References

- Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning. Springer.

- Maseer, Z. K., Yusof, R., Bahaman, N., Mostafa, S. A., and Mohd Foozy, C. F. (2021). Benchmarking of machine learning for anomaly-based intrusion detection systems in the CICIDS2017 dataset. *IEEE Access*, 9, 22351-22370.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Sharafaldin, I., Habibi Lashkari, A., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108-116.
- Canadian Institute for Cybersecurity. CICIDS2017 dataset. University of New Brunswick.